

10 Paesaggi complessi e metaeuristiche stocastiche

Antonio Scala

24 Feb 2026

Obiettivi della lezione

In Lec01 abbiamo introdotto i paesaggi di potenziale per dinamiche deterministiche continue. In Lec03 abbiamo costruito catene di Markov per campionare distribuzioni complesse. In Lec08 abbiamo visto come stimare parametri massimizzando una log-likelihood. Questa lezione affronta una situazione in cui *cercare* e *campionare* diventano difficili per le stesse ragioni: lo spazio delle configurazioni è enorme, irregolare, vincolato o discreto, e una procedura puramente locale tende a bloccarsi.

Studieremo una famiglia di strategie – le metaeuristiche stocastiche – che affrontano questo problema con meccanismi diversi: rumore controllato, raffreddamento progressivo, repliche a temperature diverse, popolazioni di soluzioni, memoria esplicita e memoria distribuita.

Al termine della lezione lo studente dovrebbe essere in grado di:

1. riconoscere quando un problema ha la struttura di una ricerca in un paesaggio complesso;
2. estendere il vocabolario di Lec01 al caso combinatorio (vicinato, mosse elementari, degenerazione);
3. distinguere ottimizzazione e campionamento;
4. spiegare il limite della discesa greedy in presenza di barriere e minimi locali;
5. costruire l'algoritmo di simulated annealing a partire da Metropolis (Lec03);
6. comprendere parallel tempering come strategia per il mixing tra modi separati;
7. descrivere genetic algorithms, tabu search e ant colony optimisation in termini dei rispettivi meccanismi di esplorazione/sfruttamento;
8. confrontare le strategie su una base concettuale comune;
9. riconoscere il ponte tra “cercare in un paesaggio” e “apprendere un paesaggio” come modello probabilistico.

Struttura

1. Perché servono metodi euristici e stocastici
 2. Paesaggi complessi: dal continuo al combinatorio
 3. Due esempi guida
 4. Ricerca locale e limiti della discesa greedy
 5. Metropolis a temperatura fissata
 6. Simulated annealing
 7. Parallel tempering
 8. Genetic algorithms
 9. Tabu search
 10. Ant colony optimisation
 11. Lettura unificante
 12. Verso la lezione successiva
-

1. Perché servono metodi euristici e stocastici

1.1 Ricerca esaustiva e crescita combinatoria

Molti problemi computazionali possono essere espressi come:

$$x^* = \arg \min_{x \in \mathcal{X}} C(x),$$

dove $\mathcal{X} = \{x_1, x_2, \dots, x_M\}$ è uno spazio di configurazioni candidate e $C(x)$ è una funzione di valutazione.

Se M è piccolo, possiamo valutare $C(x)$ su tutte le configurazioni e scegliere la migliore. In molti problemi realistici, però, M cresce in modo combinatorio: assegnazioni di turni, percorsi su grafi, sottoinsiemi di variabili, configurazioni binarie con N bit (2^N casi), partizioni di una rete. La ricerca esaustiva diventa impraticabile.

Idea chiave Non possiamo visitare tutto lo spazio delle soluzioni.
Dobbiamo scegliere dove guardare.

1.2 Limiti della discesa greedy

Una prima strategia è la ricerca locale: si parte da una soluzione iniziale x_0 , si considerano configurazioni vicine, ci si sposta verso una soluzione migliore. Una discesa greedy accetta solo miglioramenti e si ferma quando nessuna mossa locale migliora C .

Il problema centrale è che una configurazione x può essere localmente ottimale – ovvero $C(x) \leq C(y)$ per ogni y nel vicinato $\mathcal{N}(x)$ – ma globalmente subottimale: esiste z lontana con $C(z) < C(x)$. Per raggiungere z può essere necessario

attraversare configurazioni intermedie con costo più alto, cosa che una procedura greedy non può fare.

Idea chiave In molti paesaggi complessi, per trovare una soluzione migliore bisogna prima attraversare regioni temporaneamente peggiori.

1.3 Un linguaggio trasversale

La stessa quantità $C(x)$ prende nomi diversi nelle varie discipline: *costo* in ricerca operativa, *loss* in machine learning, *negative log-likelihood* in statistica (Lec08), *energia* in fisica statistica, *score negativo* in biologia computazionale, *distanza dai dati* nei problemi inversi. Il lessico cambia, ma il ruolo è lo stesso: ordinare le configurazioni da più desiderabili a meno desiderabili.

Useremo nel seguito un linguaggio flessibile, indicando $C(x)$ come “costo” o “funzione obiettivo” senza impegnarci a una specifica interpretazione disciplinare.

1.4 Quando emergono le difficoltà

Le metaeuristiche diventano utili quando uno o più dei seguenti problemi sono presenti:

- spazio combinatorio: tutte le permutazioni, tutti i sottoinsiemi, tutte le partizioni;
- molti minimi locali: paesaggio multimodale, fitting non lineare, clustering con molte partizioni plausibili;
- vincoli: non tutte le configurazioni sono ammissibili (capacità, budget, fattibilità);
- non convessità: il minimo locale non è necessariamente globale;
- gradienti assenti, costosi o poco informativi (variabili discrete, simulazioni black-box);
- dati rumorosi: $C(x)$ è stimato da campioni o simulazioni;
- compromesso esplorazione/sfruttamento: bisogna esplorare regioni nuove senza disperdere lo sforzo.

Nota. Identificabilità debole e degenerazione delle soluzioni (Lec08, discussione su identificabilità e patologie della stima) compaiono qui in forma geometrica: una valle larga e piatta corrisponde a una direzione in cui i parametri non sono distinguibili dai dati.

1.5 Euristiche e ruolo della casualità

Un metodo euristico non garantisce di trovare la soluzione globale; offre una procedura ragionevole, spesso efficace, in tempi computazionalmente accettabili. Una *metaeuristica* è uno schema generale adattabile a molte classi di problemi, purché si definiscano spazio delle soluzioni, funzione obiettivo, mosse, criteri di accettazione e parametri di controllo.

La casualità entra per quattro ragioni:

1. permette di esplorare regioni diverse in esecuzioni indipendenti;
2. può consentire di uscire da minimi locali accettando occasionalmente mosse peggiorative;
3. è naturale quando $C(x)$ è stimata e quindi rumorosa;
4. in alcuni metodi non si cerca solo il minimo ma si campiona una distribuzione (come in Lec03).

2. Paesaggi complessi: dal continuo al combinatorio

In Lec01 abbiamo introdotto i sistemi di gradiente $\dot{x} = -V'(x)$, equilibri, stabilità locale, minimi/massimi come attrattori/repulsori, e l'immagine valle/barriera/bacino di attrazione. Tutto quel vocabolario si trasferisce qui, con un'estensione: lo spazio $\mathcal{X}$ può essere discreto, combinatorio o misto, e la nozione di derivata viene rimpiazzata da quella di *vicinato*.

2.1 Configurazioni e vicinato

Una configurazione $x \in \mathcal{X}$ può essere una permutazione, un sottoinsieme, una partizione, un vettore di parametri, una rete, una strategia.

Per fare ricerca locale serve una nozione di prossimità. Definiamo il *vicinato* $\mathcal{N}(x)$ come l'insieme delle configurazioni raggiungibili da x con una *mossa elementare*. Esempi:

- TSP: scambiare due città o invertire un segmento (mossa 2-opt);
- feature selection: aggiungere o togliere una variabile;
- clustering: spostare un punto da un cluster a un altro;
- parametri continui: perturbare leggermente una componente;
- reti: aggiungere o rimuovere un arco.

Idea chiave La definizione del vicinato è una scelta di modellazione. Lo stesso paesaggio può apparire liscio o rugged a seconda di quali mosse sono ammesse.

2.2 Minimi locali, barriere, bacini

Le definizioni di Lec01 Sez. 2 si estendono in modo naturale. Un *minimo locale* è una configurazione x tale che $C(x) \leq C(y)$ per ogni $y \in \mathcal{N}(x)$; un *minimo globale* è tale che $C(x^*) \leq C(x)$ per ogni $x \in \mathcal{X}$. Una *barriera* è una regione di costo più alto che separa due regioni buone: per andare da x_0 a x_k con $C(x_k) < C(x_0)$ può essere necessario passare attraverso configurazioni intermedie con $C(x_i) > C(x_0)$. Il *bacino di attrazione* di un minimo locale, per una data dinamica di ricerca, è l'insieme delle condizioni iniziali che terminano in quel minimo.

Nota. Il bacino dipende dalla dinamica, non solo dal paesaggio. Due algoritmi sullo stesso $C(x)$ possono produrre bacini diversi perché le mosse ammesse sono diverse.

2.3 Ruggedness e degenerazione

Un paesaggio è *rugged* quando presenta molti minimi locali, barriere e variazioni rapide. Un paesaggio è *degenerato* quando molte configurazioni diverse hanno costi simili, anche senza essere particolarmente rugged. La degenerazione è particolarmente importante in problemi di inferenza: corrisponde alla non identificabilità di la lezione sulla stima dei parametri, nella discussione su identificabilità e patologie della likelihood, e cambia il significato del problema. A volte non vogliamo una singola soluzione ottima, ma la famiglia delle soluzioni quasi equivalenti. Questo collega ottimizzazione e campionamento (Sez. 11).

2.4 Esplorazione vs sfruttamento

Ogni algoritmo di ricerca bilancia due comportamenti opposti.

- *Esplorazione*: visitare regioni nuove dello spazio per evitare di rimanere confinati nel primo bacino trovato.
- *Sfruttamento*: migliorare soluzioni già promettenti per scendere verso costi più bassi.

Troppa esplorazione produce ricerca dispersiva; troppo sfruttamento produce convergenza prematura. Le diverse metaeuristiche differiscono soprattutto per il modo in cui gestiscono questo compromesso, ed è la chiave di lettura che useremo nel confronto finale (Sez. 11).

2.5 Una struttura comune in molti contesti

Lo stesso linguaggio descrive problemi molto diversi:

Contesto	Configurazione x	Funzione $C(x)$	Difficoltà tipica
Logistica/routing	percorso, sequenza	distanza, ritardi	combinatoria, vincoli
Machine learning	parametri	loss	non convessità, selle
Statistica/Bayes	parametri	$-\log p$	multimodalità, identificabilità
Reti	partizione, flusso	modularità, costo	degenerazione
Biologia comp.	configurazione, rete	score, distanza	molte soluzioni compatibili
Economia/portafogli	ipotesi, allocazione	rischio, costo	vincoli, trade-off
Modelli agent-based	parametri/regole	distanza dai dati	valutazione rumorosa

Idea chiave Molti problemi diversi diventano computazionalmente simili quando li leggiamo come esplorazione di uno spazio di configurazioni.

3. Due esempi guida

Per confrontare i diversi metodi useremo due esempi guida ricorrenti.

3.1 Esempio A – Funzione 2D multimodale

Una funzione continua di due variabili,

$$C(x, y) = a(x^2 + y^2) - \sum_{k=1}^K A_k \exp \left[-\frac{(x - x_k)^2 + (y - y_k)^2}{2s_k^2} \right],$$

con valli centrate in (x_k, y_k) , profondità A_k e larghezza s_k . Il termine quadratico confina la dinamica. Permette di costruire paesaggi controllati: una valle profonda e stretta, una valle larga e poco profonda, barriere alte o basse a piacere.

Scopo didattico: rendere visibile l'effetto della temperatura, di una traiettoria greedy, di un raffreddamento, di repliche a temperature diverse. Useremo questo esempio principalmente per i metodi termici (Sez. 5-7).

3.2 Esempio B – TSP euclideo piccolo

Generiamo N punti nel piano ($N = 20-50$), li interpretiamo come nodi (città, punti di consegna, stazioni). Una configurazione è una permutazione $\pi = (\pi_1, \dots, \pi_N)$ e la funzione obiettivo è

$$C(\pi) = \sum_{i=1}^N d(\pi_i, \pi_{i+1}), \quad \pi_{N+1} = \pi_1.$$

Mossa locale naturale: 2-opt (scegli due archi, inverti il segmento intermedio).

Scopo didattico: mostrare come funzionano le metaeuristiche su uno spazio combinatorio con vicinato discreto. Useremo questo esempio per ricerca locale, simulated annealing, tabu search, ant colony optimisation, genetic algorithms.

3.3 Un terzo esempio concettuale

Useremo occasionalmente come terzo esempio una *negative log-likelihood multimodale* $C(\theta) = -\ell(\theta)$, in continuità con Lec08. Serve come ponte tra ottimizzazione e campionamento (Sez. 7 e Sez. 12), perché lì il problema non è solo trovare un massimo ma esplorare l'incertezza sui parametri.

4. Ricerca locale e limiti della discesa greedy

4.1 Algoritmo greedy

Ingredienti: configurazione corrente x , vicinato $\mathcal{N}(x)$, criterio “accetta solo miglioramenti”.

```
scegli x iniziale
ripeti:
    genera y nel vicinato di x
    se  $C(y) < C(x)$ :
         $x \leftarrow y$ 
fino a quando nessuna mossa locale migliora C
```

Varianti: *first improvement* (accetta la prima mossa migliorativa), *best improvement* (valuta tutto il vicinato), *steepest descent* nel continuo, *random local search*.

4.2 Quando funziona

La ricerca locale è spesso sorprendentemente efficace. Nel TSP, una mossa 2-opt rimuove incroci inutili. In feature selection, togliere una variabile irrilevante migliora la generalizzazione. Nello scheduling, spostare un'attività fuori da uno slot sovraccarico riduce ritardi. La ricerca locale è un componente prezioso, non un avversario.

4.3 Tre patologie

Minimi locali. L'algoritmo si ferma in x con $C(x) \leq C(y)$ per ogni $y \in \mathcal{N}(x)$, anche se esiste z con $C(z) < C(x)$. Non può uscire perché ogni cammino verso z richiede almeno un passo peggiorativo.

Dipendenza dall'inizializzazione. Il risultato dipende dal bacino di attrazione iniziale. Strategia parziale: *random restart* (eseguire molte volte da condizioni iniziali diverse, conservare la migliore). Non basta se i bacini delle soluzioni migliori sono piccoli o difficili da raggiungere.

Incapacità di attraversare barriere. Conseguenza dei punti precedenti, ma vale la pena enunciarla in forma diretta:

Idea chiave Una regola che migliora sempre localmente può impedire miglioramenti globali. Per migliorare bisogna a volte poter peggiorare.

4.4 Cicli e oscillazioni

In una variante che accetta anche mosse non strettamente migliorative – ad esempio in presenza di costi quasi uguali o rumore nella valutazione – possono comparire cicli: una mossa viene annullata da quella successiva. Questo motiva i metodi con memoria (Sez. 9, tabu search).

4.5 Esempio: 2-opt nel TSP

Una soluzione π è *2-optimal* se nessuna singola mossa 2-opt migliora il percorso. Una soluzione 2-optimal può essere ancora lontana dall'ottimo globale: per migliorare può servire una sequenza di mosse in cui qualcuna peggiora temporaneamente.

4.6 Esempio: forward/backward feature selection

In *forward selection* si parte da nessuna variabile e si aggiunge ogni volta quella che migliora di più. In *backward elimination* si parte da tutte e si rimuove ogni volta la meno utile. Quando le variabili interagiscono (una variabile utile solo in combinazione con un'altra), entrambe le strategie possono bloccarsi in sottoinsiemi localmente buoni ma non globalmente ottimali.

Idea chiave La ricerca locale non è da scartare: è il mattone di base. Le metaeuristiche la arricchiscono con rumore, memoria, popolazioni o repliche, ma quasi tutte la usano come componente interna.

5. Metropolis a temperatura fissata

In Lec03 abbiamo costruito l'algoritmo di Metropolis come catena di Markov reversibile rispetto a una distribuzione target π . Qui lo rileggiamo come *strategia di esplorazione di un paesaggio*: la stessa formula, una prospettiva diversa.

5.1 Richiamo della regola di accettazione

Definita la variazione di costo per una mossa proposta $x \rightarrow y$,

$$\Delta C = C(y) - C(x),$$

la probabilità di accettazione (per proposta simmetrica) è

$$p_{\text{acc}}(x \rightarrow y) = \min\left(1, \exp\left[-\frac{\Delta C}{T}\right]\right).$$

Se $\Delta C \leq 0$ la mossa è sempre accettata. Se $\Delta C > 0$, la mossa è accettata con probabilità $\exp[-\Delta C/T] \in (0, 1)$.

Nota. Come dimostrato in Lec03, la regola implementa il detailed balance rispetto a $\pi_T(x) \propto \exp[-C(x)/T]$, e la costante di normalizzazione $Z(T)$ si cancella nel rapporto. È questo che rende il metodo utilizzabile anche quando $Z(T)$ è intrattabile.

5.2 La temperatura come parametro di esplorazione

Rispetto a Lec03, qui T non è un parametro fisico ma un *parametro di controllo dell'esplorazione*:

- T grande: peggioramenti moderati hanno probabilità di accettazione apprezzabile; la dinamica esplora liberamente, attraversa barriere, ma distingue poco tra soluzioni buone e cattive.
- T piccolo: peggioramenti sono rari; la dinamica diventa selettiva e tende a concentrarsi nelle valli profonde.
- $T \rightarrow 0$: si recupera la ricerca greedy.
- $T \rightarrow \infty$: si recupera una passeggiata casuale poco selettiva.

Idea chiave La temperatura controlla quanto l'algoritmo sia disposto ad accettare peggioramenti. La casualità non rende l'algoritmo meno razionale: evita che una razionalità troppo locale lo blocchi.

5.3 Metropolis non è ancora simulated annealing

Una distinzione importante:

- *Metropolis a T fissata* è un metodo di **campionamento** della distribuzione π_T . Le proprietà di mixing, autocorrelazione e metastabilità sono discusse in Lec03 Sez. 8.
- *Simulated annealing* è un metodo di **ottimizzazione** costruito facendo decrescere T nel tempo. Vedremo perché in Sez. 6.

5.4 Limiti pratici a temperatura fissata

I limiti già discussi in Lec03 ricompaiono qui in forma operativa:

- la scelta di T deve essere coerente con la scala tipica di ΔC ;
- in paesaggi con barriere molto alte, il *mixing* tra modi può essere lento (vedi Sez. 7 sul parallel tempering);
- la regola di proposta $q(y | x)$ è cruciale: mosse troppo piccole rendono lenta l'esplorazione, mosse troppo grandi vengono rifiutate spesso;
- i campioni sono correlati: l'autocorrelazione misura quanti passi servono per un campione "indipendente" (Lec03 Sez. 8).

6. Simulated annealing

6.1 Dal campionamento all'ottimizzazione

L'idea è semplice: usare Metropolis, ma far decrescere T nel tempo. Si parte da una temperatura alta T_0 e si costruisce una sequenza

$$T_0 > T_1 > T_2 > \dots > T_K.$$

A temperatura alta l'algoritmo esplora ampiamente. A temperatura intermedia attraversa selettivamente alcune barriere. A temperatura bassa raffina localmente. Si trasforma quindi un metodo di campionamento in una strategia di ottimizzazione approssimata.

Il termine “annealing” viene dalla metallurgia (riscaldamento e raffreddamento lento) ed è solo una metafora computazionale.

6.2 Regola di accettazione

A ogni passo n :

$$p_{\text{acc}}(x \rightarrow y) = \min\left(1, \exp\left[-\frac{\Delta C}{T_n}\right]\right).$$

Lo stesso peggioramento ΔC diventa progressivamente meno accettabile.

scegli x iniziale, fissa $T = T_0$, scegli cooling schedule
ripeti:

```

    proponi  $x \rightarrow y$  (mossa locale)
    DeltaC =  $C(y) - C(x)$ 
    se DeltaC <= 0: accetta  $y$ 
    altrimenti accetta  $y$  con probabilita  $\exp(-\text{DeltaC}/T)$ 
    se accettata:  $x \leftarrow y$ 
    aggiorna  $T$  secondo lo schedule
    se  $C(x) < C_{\text{best}}$ :  $x_{\text{best}} \leftarrow x$ 
fino al criterio di arresto
restituisce  $x_{\text{best}}$ 
```

Nota. Si restituisce di solito la migliore configurazione visitata, non l'ultima. A temperatura non nulla la traiettoria può temporaneamente allontanarsi da una buona soluzione.

6.3 Cooling schedule

Le scelte tipiche sono:

$$T_{n+1} = \alpha T_n, \quad 0 < \alpha < 1 \quad (\text{geometrico});$$

$$T_n = \frac{T_0}{1 + \beta n}, \quad \beta > 0 \quad (\text{iperbolico});$$

$$T_n = \frac{T_0}{\log(n + n_0)} \quad (\text{logaritmico}).$$

Il raffreddamento logaritmico ha garanzie teoriche di convergenza ma è spesso troppo lento in pratica. Il geometrico è il più usato.

Idea chiave Il cooling schedule è un compromesso. Raffreddamento troppo rapido: l'algoritmo diventa presto greedy e si blocca. Raffreddamento troppo lento: costo computazionale eccessivo.

6.4 Temperatura iniziale e finale

Per stimare T_0 si può chiedere che un peggioramento tipico ΔC_{typ} sia accettato con probabilità target p (es. $p = 0.5$):

$$T_0 \approx -\frac{\Delta C_{\text{typ}}}{\log p}.$$

La temperatura finale dovrebbe essere abbastanza bassa da rendere i peggioramenti rari, in modo che la fase finale operi come un raffinamento locale.

6.5 Plateau di temperatura

Spesso conviene tenere T fissa per M passi prima di abbassarla:

```
per ogni T_k:  
  esegui M passi Metropolis a T_k  
  abbassa T
```

Questo permette alla dinamica di esplorare la distribuzione associata a ogni temperatura prima del passo successivo. Il numero M dipende dal mixing della catena a quella temperatura.

6.6 Esempi

TSP (esempio B). Una mossa 2-opt che allunga il percorso può essere accettata con probabilità $\exp[-\Delta C/T]$. Fase calda: riorganizzazione ampia; fase intermedia: attraversamento di strutture localmente subottimali; fase fredda: raffinamento del percorso.

Scheduling. Spostare un'attività può peggiorare temporaneamente il piano ma sbloccare una riorganizzazione successiva. Simulated annealing permette questi cambiamenti coordinati che una ricerca greedy rifiuterebbe.

Calibrazione di modelli. Quando $C(\theta)$ ha molti minimi locali (la lezione sulla stima dei parametri, nella discussione su minimi locali e patologie della likelihood), simulated annealing è utile come *fase esplorativa iniziale*, seguita poi da un metodo locale più preciso. Combinazione tipica: SA globale + ottimizzazione locale + diagnostica statistica.

6.7 Parametri e diagnostiche

Le scelte critiche sono: T_0 , T_K , forma dello schedule, M passi per temperatura, tipo di mossa locale, criterio di arresto. Quantità da monitorare:

- costo corrente e migliore costo trovato;
- tasso di accettazione globale e per le mosse peggiorative;
- variabilità tra run indipendenti con seed diversi;
- confronto con baseline (ricerca greedy con restart, random search).

Se SA non batte una baseline semplice, è probabilmente mal calibrato.

7. Parallel tempering

7.1 Il limite di una singola traiettoria

Simulated annealing segue una sola traiettoria. Se questa entra precocemente in un bacino locale, può non uscirne più. In paesaggi *fortemente multimodali* (più regioni plausibili separate da barriere alte), una sola traiettoria è strutturalmente inadatta.

Parallel tempering (o *replica exchange*) affronta il problema in modo diverso: invece di far decrescere T nel tempo, mantiene contemporaneamente R repliche a temperature diverse.

7.2 Repliche a temperature diverse

Scegliamo una scala

$$T_1 < T_2 < \dots < T_R.$$

Ogni replica r evolve via Metropolis alla propria temperatura T_r e, vista isolatamente, campionerebbe

$$\pi_{T_r}(x) \propto \exp\left[-\frac{C(x)}{T_r}\right].$$

Le repliche fredde sono concentrate sulle valli profonde; le calde attraversano facilmente le barriere.

7.3 Scambi tra repliche

Periodicamente si propone di scambiare le configurazioni di due repliche, di solito a temperature adiacenti. Per due repliche i, j con configurazioni x_i, x_j , lo scambio è accettato con

$$p_{\text{swap}} = \min\left(1, \exp\left[\left(\frac{1}{T_i} - \frac{1}{T_j}\right)(C(x_i) - C(x_j))\right]\right).$$

Nota. La formula segue dal detailed balance applicato alla distribuzione prodotto sulle repliche (Lec03 Sez. 6). Quando si scambiano

x_i e x_j cambiano solo i due fattori $\exp[-C(x_r)/T_r]$ relativi a i e j ; il rapporto fornisce il fattore esponenziale.

Gli scambi permettono alle configurazioni di *viaggiare lungo la scala delle temperature*: una configurazione generata in una replica calda può attraversare una barriera, essere trasferita verso temperature più basse e poi raffinata; viceversa una replica fredda intrappolata può “risalire” e cambiare bacino.

7.4 Campionamento multimodale

Parallel tempering nasce come metodo di **campionamento**, non di ottimizzazione pura. È utile soprattutto quando la distribuzione target è multimodale e una singola catena MCMC a bassa temperatura ha mixing lento (problema discusso in Lec03 Sez. 8.5).

Applicazione tipica: posterior bayesiana multimodale. Definendo

$$C(\theta) = -\log p(D \mid \theta) - \log p(\theta),$$

la posterior diventa $p(\theta \mid D) \propto \exp[-C(\theta)]$. Le repliche calde appiattiscono la posterior; gli scambi permettono alla catena fredda di visitare più regioni plausibili. Questo è importante per **rappresentare correttamente l'incertezza** quando esistono spiegazioni multiple dei dati (collegamento a la lezione sulla stima dei parametri, nella discussione su identificabilità e patologie della likelihood sull'identificabilità).

7.5 Scelta delle temperature

Se le T_r sono troppo distanti, gli scambi vengono rifiutati spesso e le repliche non comunicano. Se sono troppo vicine, servono molte repliche per coprire l'intervallo utile e il costo aumenta. Una progressione geometrica $T_r = T_1 a^{r-1}$ è una scelta comune; una regola pratica è scegliere temperature tali da ottenere scambi non rari tra repliche adiacenti; in molte applicazioni si mira a tassi dell'ordine del 20–40%, ma il valore ottimale dipende dal problema.

7.6 Diagnostiche pratiche

- tasso di accettazione degli scambi;
- *round-trip*: una configurazione deve poter viaggiare dalla temperatura più alta alla più bassa e tornare indietro;
- variabilità tra run indipendenti;
- autocorrelazione e mixing in ciascuna replica.

7.7 Esempi

Posterior bayesiana multimodale (già discusso sopra).

Modelli non identificabili / valli piatte. In modelli epidemiologici, agent-based o biologici, parametri diversi possono produrre previsioni quasi indistinguibili. Una singola ottimizzazione locale restituisce un valore ma nasconde le alternative; parallel tempering esplora la varietà di soluzioni plausibili.

Clustering e community detection. Molte partizioni possono avere qualità simile. Le repliche calde permettono cambiamenti strutturali ampi nella partizione; le repliche fredde le valutano in modo selettivo. Importante per evitare di presentare una sola partizione come “la” struttura dei dati.

7.8 Confronto con simulated annealing

	simulated annealing	parallel tempering
traiettorie	una	R in parallelo
temperatura	decresce nel tempo	fissa, diversa per replica
obiettivo primario	ottimizzazione	campionamento multimodale
esplorazione calda	scompare nel tempo	sempre attiva
costo	basso per iterazione	maggiore, parallelizzabile

Idea chiave Simulated annealing raffredda una traiettoria; parallel tempering fa cooperare traiettorie a temperature diverse. Non sempre conviene scegliere tra esplorazione e sfruttamento in momenti diversi; a volte conviene tenerli simultaneamente in repliche diverse.

8. Genetic algorithms

8.1 Dalla traiettoria alla popolazione

Finora abbiamo discusso metodi che seguono una o più *traiettorie* nello spazio delle configurazioni. I genetic algorithms cambiano punto di vista: mantengono una *popolazione* di soluzioni candidate,

$$\mathcal{P}^{(g)} = \{x_1^{(g)}, \dots, x_N^{(g)}\},$$

e la fanno evolvere tramite *selezione*, *crossover* e *mutazione*. Il linguaggio è ispirato all'evoluzione biologica, ma l'interpretazione è computazionale: la popolazione mantiene esplicitamente la diversità delle soluzioni, e la diversità è una risorsa.

8.2 Rappresentazione e fitness

Una soluzione (o *cromosoma*) può essere:

- stringa binaria (feature selection: $x_j = 1$ se la variabile j è inclusa);
- vettore reale (parametri continui);

- permutazione (TSP);
- struttura mista discreta/continua (portafogli con scelta di asset + pesi);
- oggetto strutturato (rete, partizione).

La *fitness* $F(x)$ misura quanto una soluzione sia promettente; in problemi di minimizzazione si può prendere $F(x) = -C(x)$, $F(x) = 1/(1 + C(x))$ o trasformazioni analoghe.

Nota. Una buona codifica delle soluzioni è spesso più importante della scelta dei parametri dell'algoritmo. Una codifica che rispetta la struttura del problema rende naturali crossover e mutazioni. La flessibilità dei GA è anche il loro rischio: se codifica, crossover e mutazione non incorporano la struttura del problema, l'algoritmo può ridursi a una ricerca casuale costosa.

8.3 Schema generale

```

inizializza popolazione P di N soluzioni
valuta la fitness di ogni soluzione
ripeti:
    seleziona genitori in base alla fitness
    applica crossover su coppie selezionate
    applica mutazione casuale su alcuni individui
    valuta i nuovi individui
    componi la nuova generazione (eventualmente con elitismo)
fino al criterio di arresto
restituisce la migliore soluzione trovata

```

8.4 Selezione

Bilancia *pressione selettiva* (favorire le soluzioni migliori) e *diversità* (mantenere alternative esplorative).

- *Proporzionale alla fitness*: $P(i) = F(x_i) / \sum_j F(x_j)$. Semplice, ma sensibile alla scala.
- *Tournament selection*: si pescano k individui a caso, si sceglie il migliore. Il parametro k regola la pressione selettiva.
- *Rank selection*: probabilità basata sul rango, non sul valore assoluto della fitness. Robusta alla scala.

8.5 Crossover

Combina due genitori. La forma dipende dalla codifica.

- *Stringhe binarie*: single-point crossover. Dato k , il figlio prende $(x_1, \dots, x_k, z_{k+1}, \dots, z_d)$.
- *Vettori reali*: $y = \lambda x + (1 - \lambda)z$ con $\lambda \in [0, 1]$.

- *Permutazioni* (TSP): servono operatori specifici (Order Crossover, PMX, ecc.) che producano permutazioni valide senza duplicati.

Idea chiave Il crossover deve rispettare la struttura dello spazio delle soluzioni. Un crossover “naive” su permutazioni produce soluzioni inammissibili.

8.6 Mutazione ed elitismo

La *mutazione* introduce variazioni casuali (flip di bit, perturbazione gaussiana, scambio di due elementi, modifica di un arco). Mantiene la diversità e permette di visitare regioni non rappresentate nella popolazione. Il tasso di mutazione è un parametro delicato: troppo basso porta a convergenza prematura, troppo alto a ricerca quasi casuale.

L'*elitismo* conserva esplicitamente i migliori individui da una generazione all'altra. Evita di perdere le migliori soluzioni per effetto di operatori sfavorevoli; un eccesso di elitismo riduce la diversità.

8.7 Esplorazione/sfruttamento distribuiti

Diversamente dai metodi termici, in un GA il compromesso esplorazione/sfruttamento è distribuito tra più componenti:

- selezione ed elitismo → sfruttamento;
- mutazione e diversità della popolazione → esplorazione;
- crossover ha ruolo intermedio: ricombina materiale già presente.

8.8 Convergenza prematura

Patologia tipica: la popolazione diventa rapidamente omogenea intorno a una soluzione localmente buona. Segnali: fitness media che converge alla migliore, individui quasi identici, miglioramenti che si fermano. Cause: selezione troppo forte, popolazione troppo piccola, mutazione troppo bassa, elitismo eccessivo. Rimedi: aumentare la popolazione, ridurre la pressione selettiva, tassi di mutazione adattivi, niching, restart parziali, ibridazione con ricerca locale.

8.9 Quando un GA è naturale

- Spazi misti discreto-continuo (es. portafogli con scelta degli asset + pesi).
- Codifiche strutturate (reti, sequenze, regole).
- Funzione obiettivo black-box, costosa, non differenziabile (modelli agent-based: la fitness è una statistica di simulazione, eventualmente rumorosa).
- Possibilità di parallelizzare: valutare gli individui di una generazione in parallelo.

8.10 Ibridazione con ricerca locale (memetic algorithms)

Spesso conviene applicare una procedura locale (es. 2-opt nel TSP) a ciascun individuo dopo crossover e mutazione, prima della valutazione. La popolazione esplora; la ricerca locale raffina. Questo conferma il ruolo della ricerca locale come componente, non come avversaria.

9. Tabu search

9.1 Memoria al posto del rumore

Una ricerca locale che accetta anche mosse non strettamente migliorative tende a oscillare: una mossa viene annullata da quella successiva. La risposta di tabu search non è introdurre rumore, ma **memoria**: alcune mosse o attributi vengono dichiarati *tabu* (proibiti) per un certo numero di passi.

Idea chiave Tabu search usa memoria *negativa* (vietare ritorni); ant colony (Sez. 10) userà memoria *positiva* (rinforzare componenti buone); genetic algorithms (Sez. 8) usano memoria *implicita* nella popolazione. Tre strategie diverse per risolvere lo stesso problema della ricerca locale.

9.2 Lista tabu e tabu tenure

La *lista tabu* contiene mosse, configurazioni o attributi proibiti temporaneamente. Memorizzare *configurazioni* è semplice ma costoso. Più spesso si memorizzano *attributi* di una mossa: nel TSP gli archi rimossi o aggiunti; nello scheduling l'assegnazione attività-slot.

La *tabu tenure* τ è il numero di passi per cui un attributo resta proibito. Troppo piccola: memoria debole, la ricerca torna sui propri passi. Troppo grande: troppe mosse vietate, ricerca eccessivamente vincolata. Può essere costante, casuale in un intervallo, o adattiva.

9.3 Algoritmo

```
scegli x iniziale, lista tabu vuota, x_best = x
ripeti:
    genera il vicinato N(x), rimuovi mosse tabu
    scegli la migliore mossa ammissibile x -> y
    x <- y      (anche se C(y) > C(x))
    aggiorna la lista tabu (aggiungi attributo, scarta i vecchi)
    se C(x) < C(x_best): x_best <- x
fino al criterio di arresto
restituisce x_best
```

Due punti chiave:

1. la mossa scelta **può peggiorare** il costo. La ricerca non si ferma quando il greedy si fermerebbe.
2. La logica non è probabilistica: si sceglie la migliore alternativa disponibile, ma alcune sono vietate dalla memoria.

9.4 Criterio di aspirazione

Una mossa tabu può essere ammessa se conduce a una soluzione molto buona, tipicamente migliore della migliore globale finora trovata:

se $C(y) < C(x_{\text{best}})$, ammetti y anche se tabu.

Evita che la memoria diventi troppo rigida e blocchi soluzioni eccellenti.

9.5 Intensificazione e diversificazione

Oltre alla memoria a breve termine (lista tabu), tabu search può usare *memoria a lungo termine*:

- *intensificazione*: favorire configurazioni che mantengono attributi ricorrenti nelle soluzioni migliori (sfruttamento);
- *diversificazione*: penalizzare attributi troppo frequenti per spingere la ricerca in regioni nuove (esplorazione).

Idea chiave La memoria modifica temporaneamente il paesaggio percepito dall'algoritmo. La funzione $C(x)$ non cambia, ma cambia quali mosse sono ammissibili in un dato momento.

9.6 Esempi

TSP. Dopo una mossa 2-opt, vietare temporaneamente di reinserire gli archi rimossi o di rimuovere quelli aggiunti. Forza la ricerca a esplorare riorganizzazioni diverse.

Scheduling e assegnamento. Vietare di riportare un'attività allo slot precedente, o di annullare uno scambio appena fatto. Evita cicli brevi e oscillazioni.

Vehicle routing. Vietare temporaneamente di rispostare un cliente al percorso precedente; combinato con intensificazione (preservare gruppi di clienti spesso serviti insieme) e diversificazione (penalizzare assegnazioni troppo frequenti).

Ottimizzazione su grafi. Per partizionamento, selezione di nodi, design di rete: vietare di toccare attributi recentemente modificati.

9.7 Diagnostiche

- migliore costo trovato (non l'ultimo);
- andamento del costo corrente (può oscillare, va bene);

- frazione di mosse tabu nel vicinato (se è troppo alta, la lista è troppo lunga);
- sensibilità a τ ;
- visite a regioni diverse del paesaggio (segnale di diversificazione adeguata).

10. Ant colony optimisation

10.1 Memoria distribuita

In tabu search la memoria è centralizzata (una lista). In ant colony optimisation (ACO) la memoria è **distribuita nell'ambiente**: molti agenti costruiscono soluzioni, le componenti delle soluzioni buone ricevono una traccia (“feromone”) che aumenta la probabilità di essere riutilizzate.

L'idea biologica di partenza (formiche che trovano cammini brevi tramite tracce chimiche) è solo una metafora. Il principio computazionale è la *stigmergia*: gli agenti non comunicano direttamente, ma modificano l'ambiente, e l'ambiente influenza le scelte successive.

10.2 Ingredienti

- *Agenti* che costruiscono una soluzione passo dopo passo.
- *Feromone* τ_{ij} associato a componenti della soluzione (es. archi in un grafo).
- *Informazione euristica locale* η_{ij} (es. $1/d_{ij}$ nel TSP).
- *Regola probabilistica di scelta*: in un nodo i , la probabilità di scegliere j tra le opzioni $\mathcal{A}(i)$ è

$$P_{ij} = \frac{\tau_{ij}^\alpha \eta_{ij}^\beta}{\sum_{k \in \mathcal{A}(i)} \tau_{ik}^\alpha \eta_{ik}^\beta}.$$

I parametri $\alpha, \beta \geq 0$ regolano il peso relativo di feromone (memoria collettiva) ed euristica (convenienza locale). In pratica i feromoni vengono inizializzati a valori positivi, per evitare che alcune componenti abbiano probabilità nulla fin dall'inizio e non possano più essere esplorate quando $\alpha > 0$.

10.3 Aggiornamento del feromone

Dopo che gli agenti hanno costruito le proprie soluzioni:

$$\tau_{ij} \leftarrow (1 - \rho) \tau_{ij} + \Delta \tau_{ij}, \quad 0 < \rho < 1.$$

Due meccanismi opposti:

- *Evaporazione* $(1 - \rho)\tau_{ij}$: la memoria delle scelte vecchie svanisce. Senza evaporazione, le prime tracce dominerebbero per sempre. ρ troppo piccolo: cristallizzazione; ρ troppo grande: memoria debole.

- *Rinforzo* $\Delta\tau_{ij}$: le buone soluzioni depositano feromone sulle componenti che le costituiscono. Nel TSP, ad esempio,

$$\Delta\tau_{ij}^{(a)} = \begin{cases} Q/L_a & \text{se l'agente } a \text{ ha usato l'arco } (i, j), \\ 0 & \text{altrimenti,} \end{cases}$$

con L_a lunghezza del percorso dell'agente a . Percorsi più brevi rinforzano di più.

10.4 Schema generale

```

inizializza i feromoni
ripeti:
    per ogni agente:
        costruisci una soluzione passo per passo
        usando P_ij dipendente da feromone ed euristica
    valuta le soluzioni
    evapora il feromone
    deposita feromone sulle componenti delle soluzioni buone
    aggiorna x_best
fino al criterio di arresto
restituisce x_best

```

Idea chiave Le metaeuristiche usano la memoria in modi diversi: tabu (negativa, centralizzata), GA (implicita, popolazionale), ACO (positiva, distribuita). Tutti modificano la dinamica della ricerca usando informazione sulla storia.

10.5 Quando ACO è naturale

- Problemi su grafi e costruzioni di cammini (shortest path, TSP, vehicle routing).
- Routing in reti dinamiche (latenza, congestione, affidabilità variabili nel tempo: l'evaporazione permette adattamento).
- Logistica: consegne, raccolta, manutenzione, ispezioni.
- Problemi in cui la soluzione si costruisce naturalmente come sequenza di scelte locali.

10.6 Stagnazione e ibridazione

Patologia tipica: il feromone si concentra rapidamente su poche componenti; gli agenti costruiscono soluzioni quasi identiche; l'esplorazione si riduce. Contromisure: limiti minimi e massimi al feromone (varianti MAX-MIN), evaporazione più forte, rinforzo meno aggressivo, ricerca locale integrata.

Come per i GA, ACO è spesso ibridato con ricerca locale (es. 2-opt nel TSP): l'agente costruisce un percorso, lo migliora localmente, e solo dopo si aggiorna il feromone.

10.7 Diagnostiche

- migliore soluzione trovata;
- distribuzione del feromone sugli archi (concentrato vs uniforme);
- diversità delle soluzioni costruite;
- sensibilità a α, β, ρ ;
- variabilità tra run.

11. Lettura unificante

Le metaeuristiche non sono una collezione di trucchi separati. Sono modi diversi di affrontare un problema comune: esplorare uno spazio complesso bilanciando esplorazione e sfruttamento.

11.1 Tabella comparativa

Metodo	Meccanismo principale	Oggetto mantenuto	Più naturale per	Rischio principale
Metropolis (T fissa)	rumore termico	una catena	campionamento	mixing lento
Simulated annealing	T decrescente	una traiettoria	ottimizzazione	schedule mal calibrato
Parallel tempering	repliche a T diverse	molte catene	campionamento multimodale	scambi rifiutati
Genetic algorithms	selezione + crossover + mutazione	popolazione	spazi misti discreto/continuo	convergenza prematura
Tabu search	memoria esplicita (negativa)	traiettoria + lista	problemi combinatori	tenure mal calibrata
Ant colony	memoria distribuita (positiva)	agenti + feromone	grafi e cammini	stagnazione

11.2 Cinque dimensioni di confronto

Singola traiettoria vs popolazione. Metropolis, SA e tabu search seguono una traiettoria; parallel tempering mantiene R traiettorie comunicanti; GA e ACO mantengono molte soluzioni o molti agenti.

Rumore vs memoria vs popolazione. Tre meccanismi distinti per evitare il blocco locale: - *rumore controllato*: SA, Metropolis; - *memoria*: tabu (centralizzata, negativa), ACO (distribuita, positiva); - *diversità della popolazione*: GA.

Esplorazione e sfruttamento. Tutti i metodi devono bilanciarli, ma il meccanismo è diverso:

Metodo	Esplorazione	Sfruttamento
SA	mosse peggiorative a T alta	T bassa $\rightarrow$ greedy
Parallel tempering	repliche calde, scambi	repliche fredde
GA	mutazione, popolazione grande	selezione, elitismo
Tabu	diversificazione	intensificazione
ACO	α basso, ρ alto	rinforzo, α alto

Ottimizzazione vs campionamento. Metropolis e parallel tempering nascono come metodi di **campionamento** (interesse: distribuzione, incertezza, modi multipli). SA, GA, tabu, ACO sono soprattutto metodi di **ottimizzazione** (interesse: una buona soluzione). La distinzione non è rigida – si possono usare metodi di campionamento per trovare il minimo, e metodi di ottimizzazione per esplorare famiglie di soluzioni – ma la domanda iniziale è diversa:

“Mi interessa una singola soluzione buona, oppure l’insieme delle soluzioni plausibili?”

Continuo vs discreto.

- *Continuo black-box o senza gradienti:* SA, Metropolis e parallel tempering.
- *Continuo con gradienti disponibili e distribuzioni ad alta dimensione:* HMC, nella lezione successiva.
- *Combinatorio puro:* SA, tabu, ACO, GA con codifiche adeguate.
- *Misto discreto-continuo:* GA e strategie ibride sono particolarmente flessibili.
- *Grafi e cammini:* ACO, tabu search e SA sono naturali.

11.3 Quale metodo scegliere

Non esiste un metodo migliore in assoluto. Domande guida:

1. **Struttura dello spazio:** continuo, combinatorio, grafo, misto?
2. **Costo di valutazione:** $C(x)$ economica o costosa? Posso parallelizzare?
3. **Multimodalità:** una soluzione basta, o serve esplorare più regioni plausibili?
4. **Identificabilità** (la lezione sulla stima dei parametri, nella discussione su identificabilità e patologie della likelihood): mi aspetto soluzioni quasi equivalenti?
5. **Rumore:** $C(x)$ è deterministica o stimata?
6. **Vincoli:** come si esprimono nelle mosse?

11.4 Errori comuni

- Usare una metaeuristica sofisticata senza confrontarla con una baseline semplice (ricerca greedy, random restart, euristica problem-specific).
- Confondere ottimizzazione e campionamento: trovare un buon x non garantisce di aver esplorato la distribuzione delle soluzioni plausibili.
- Interpretare una singola soluzione come unica in problemi con paesaggio degenerato.
- Trascurare i parametri: cooling schedule, tabu tenure, α, β, ρ in ACO, popolazione/mutazione nei GA. Non sono dettagli ma parte dell'algoritmo.
- Non verificare la stabilità tra run indipendenti.
- Prendere le metafore biologiche/fisiche troppo alla lettera. Una metaeuristica funziona quando funziona perché mantiene diversità, esplora componenti promettenti, evita ritorni banali – non perché imita la natura.

11.5 Una lettura unificante

Tutti i metodi discussi possono essere visti come modi di **modificare una ricerca locale**.

	Ricerca locale dice...
greedy	“guarda vicino e accetta solo miglioramenti”
SA	“...ma accetta peggioramenti controllati dalla temperatura”
parallel tempering	“esegui ricerche a più T e fai comunicare le repliche”
GA	“non seguire una sola ricerca, mantieni molte soluzioni che evolvono”
tabu	“continua a muoverti, ma ricorda dove sei stato”
ACO	“fai costruire soluzioni a molti agenti, lascia che le buone modifichino l'ambiente”

Idea chiave Non bisogna chiedersi quale algoritmo sia migliore in astratto, ma quale meccanismo di esplorazione sia adatto alla struttura del problema.

Nota per il laboratorio. In laboratorio non è necessario implementare tutte le metaeuristiche. È più utile confrontare poche strategie sullo stesso problema, osservando traiettorie, qualità delle soluzioni, sensibilità ai parametri e stabilità tra run indipendenti.

11.6 Quando una soluzione non basta

In paesaggi degenerati molte configurazioni hanno costi simili. Cambia la domanda:

- non solo “qual è la soluzione migliore?”;
- ma anche “quali altre soluzioni sono quasi equivalenti?”;
- “quanto è robusta la soluzione?”;
- “quali componenti sono stabili tra soluzioni diverse?”;
- “quale incertezza rimane?”

Questa è una delle ragioni per cui esistono i metodi di campionamento, e perché la prossima lezione introdurrà modelli probabilistici basati su paesaggi.

12. Verso la lezione successiva

Oggi abbiamo studiato come **esplorare** paesaggi complessi quando il paesaggio è dato. Nella prossima lezione studieremo come un paesaggio possa diventare un **modello probabilistico** e, in alcuni casi, essere **appreso dai dati**.

12.1 Da $C(x)$ a $P(x)$

Una funzione di costo $C(x)$ non serve solo a ordinare le configurazioni. Può anche definire una distribuzione di probabilità:

$$P_T(x) = \frac{1}{Z(T)} \exp\left[-\frac{C(x)}{T}\right], \quad Z(T) = \sum_x \exp\left[-\frac{C(x)}{T}\right]$$

(o l’analogo integrale nel continuo). Le configurazioni a costo basso diventano più probabili, ma quelle a costo alto non sono impossibili. Spesso, nei modelli probabilistici, si assorbe il parametro di temperatura nella definizione dell’energia oppure si pone formalmente $T = 1$:

$$P(x) = \frac{1}{Z} \exp[-E(x)].$$

La stessa funzione di costo o energia può quindi essere usata per due domande diverse:

- **ottimizzazione:** $x^* = \arg \min_x C(x)$;
- **campionamento:** generare $x \sim P(x) \propto e^{-C(x)}$.

12.2 Apprendere il paesaggio

Negli *energy-based models* la funzione di energia dipende da parametri:

$$P(x; \theta) = \frac{1}{Z(\theta)} \exp[-E(x; \theta)].$$

L'apprendimento consiste nel modificare θ affinché i dati osservati abbiano energia bassa, cioè siano probabili sotto il modello. Il paesaggio non è più dato: viene costruito dai dati.

Idea chiave Oggi abbiamo studiato come muoversi in un paesaggio dato. Nella prossima lezione vedremo paesaggi che vengono *appresi* per rappresentare i dati.

12.3 Anticipazione

I temi della prossima lezione, tutti collegati al vocabolario di oggi:

- *Hopfield networks*: dinamica deterministica con paesaggio di energia, attrattori come memorie;
- *Boltzmann Machines*: modelli energetici stocastici con variabili nascoste;
- *Gibbs sampling* (già visto in Lec03) come componente di base nell'addestramento;
- *Contrastive divergence*: il campionamento diventa parte dell'algoritmo di apprendimento, non solo strumento di analisi;
- *Hamiltonian Monte Carlo*: campionamento di distribuzioni continue ad alta dimensione, naturale in inferenza bayesiana.

In tutti questi casi torneranno le difficoltà di oggi – barriere, multimodalità, mixing lento, identificabilità – ma in un nuovo ruolo: non più ostacoli alla ricerca, ma proprietà del modello che si vuole apprendere.

Riferimenti

- Kirkpatrick, S., Gelatt, C. D., Vecchi, M. P. (1983). *Optimization by Simulated Annealing*. Science.
- Geman, S., Geman, D. (1984). *Stochastic Relaxation, Gibbs Distributions, and the Bayesian Restoration of Images*.
- Swendsen, R. H., Wang, J. S. (1986). *Replica Monte Carlo Simulation of Spin Glasses*.
- Hukushima, K., Nemoto, K. (1996). *Exchange Monte Carlo Method and Application to Spin Glass Simulations*.
- Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*.
- Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*.
- Glover, F. (1989, 1990). *Tabu Search, Parts I and II*. ORSA Journal on Computing.
- Dorigo, M., Stuetzle, T. (2004). *Ant Colony Optimization*. MIT Press.

- Robert, C. P., Casella, G. (2004). *Monte Carlo Statistical Methods*. Springer.